

Part 1 — ONNX Export and Verification

TG: P99 Tail Latency for Tabular ML Serving *David Londres — CIn/UFPE Date: 2026-05-03*

Context

Part 1 produces a single artifact, `model.onnx`, that is the contract between training and serving. Both serving stacks under comparison (FastAPI + ONNX Runtime, and BentoML) load this exact file, so any latency difference measured later is attributable to the serving stack, not to the model itself.

Dataset and split

HIGGS (UCI ML Repository): 11,000,000 rows, 28 numeric features, binary label (signal vs. background from a Higgs-boson Monte Carlo simulation). The split follows the canonical convention introduced with the dataset by Baldi et al. (2014): the first 10,500,000 rows for training and the last 500,000 rows for held-out testing — no shuffling — so the result reported here is directly comparable to the literature baseline.

Hyperparameters

Parameter	Value	Rationale
<code>n_estimators</code>	500	Target ensemble depth for the experiment
<code>max_depth</code>	8	Standard for tabular boosting; bias/variance balance
<code>learning_rate</code>	0.1	XGBoost default; not tuned
<code>tree_method</code>	<code>hist</code>	Histogram-based splits, CPU-friendly
<code>objective</code>	<code>binary:logistic</code>	Binary classification with sigmoid output
<code>random_state</code>	42	Reproducibility

These hyperparameters are frozen for the remainder of the experiment. Latency is the variable under study, so the model itself stays fixed across all serving comparisons.

Result on the 500k-row test split: AUC = 0.841, accuracy = 0.757. Training took ~4 minutes on the local CPU.

Exported graph

The exported model is a single `TreeEnsembleClassifier` operator — containing the entire 500-tree forest packed as attribute arrays — between a `[?, 28]` float32 input and two outputs: probabilities (float32 `[?, 2]`) and label (int64 `[?]`, the argmax). The serving stacks consume probabilities.

Parity check

To guarantee that the ONNX file represents the same model that was trained, predictions from XGBoost and from ONNX Runtime were compared on a fixed sample of 10,000 held-out rows. The maximum absolute difference between the two probability vectors was 4.17×10^{-7} , well below the 1×10^{-5} tolerance adopted for the check. The result is recorded in `artifacts/schema.json` alongside the SHA-256 of `model.onnx`, which the serving stacks verify at load time.

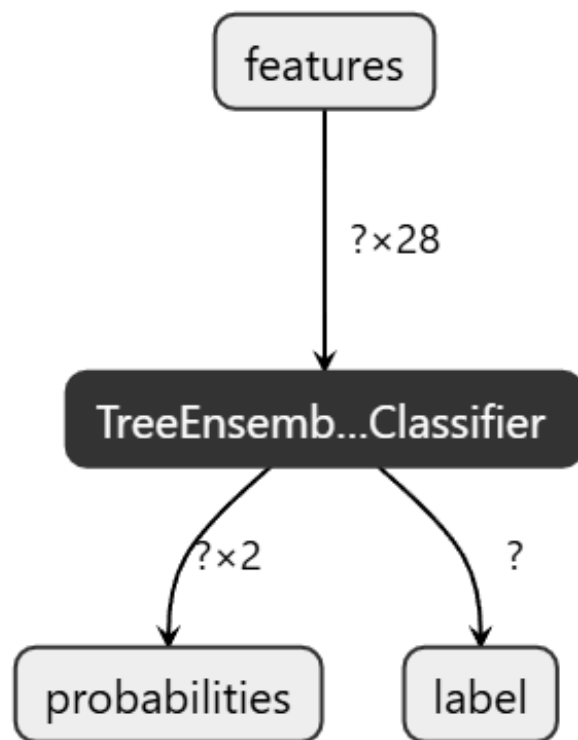


Figure 1: ONNX graph of model.onnx as rendered by Netron.